

Lecture 15: RNN & LSTM — Complete Study Guide

CSE445 Machine Learning — Full Concept Breakdown + Probable Q&A

1. Why Sequences Need a Different Kind of Network

Regular feedforward networks (like plain MLPs or CNNs used for single-image classification) take **one fixed-size input** and produce **one fixed-size output**. They have no way to relate one input to another input that came before or after it.

But many real problems are about **sequences**, where order and context matter:

Setup	Description	Example
One-to-one	Fixed input → fixed output	Plain image classification
One-to-many	Single input → sequence of outputs	Image Captioning (image → sequence of words)
Many-to-one	Sequence input → single output	Action prediction (sequence of video frames → action class)
Many-to-many (unequal length, encoder-decoder style)	Sequence in → sequence out	Video Captioning (sequence of frames → caption)
Many-to-many (aligned, same length)	Sequence in → sequence out, one per step	Video classification at the frame level

Key idea: we need an architecture that can accept variable-length sequences and carry information from earlier positions forward. That's what RNNs are built for.

Likely exam question: "Give an example each of one-to-many and many-to-one sequence processing." **Answer:** One-to-many → Image captioning (one image produces a sequence of words). Many-to-one → Action prediction (a sequence of video frames maps to a single action class).

2. Recurrent Neural Networks (RNNs) — Core Idea

How RNNs differ from standard neural networks:

Standard NNs process data only **forward**: input $\rightarrow$ output, with no memory of past inputs. RNNs introduce **feedback connections** — information cycles back into the network. At every time step t , the RNN takes **two inputs**: the current input x_t and the **hidden state** from the previous step h_{t-1} (this hidden state acts as memory).

Intuition — predicting the next word: When you read a sentence, you use the words you've already seen to guess the next word. RNNs mimic this:

The output of one step feeds into the next step's computation.

This lets the network build **context-aware** predictions.

This is what allows an RNN to understand sequence *structure*, not just isolated data points.

Likely exam question: "How does an RNN differ fundamentally from a standard feedforward neural network?" **Answer:** A feedforward network only maps input $\rightarrow$ output with no memory; an RNN has feedback connections so that at each time step, it combines the current input with a hidden state carrying information from all previous time steps, enabling it to model sequential/temporal dependencies.

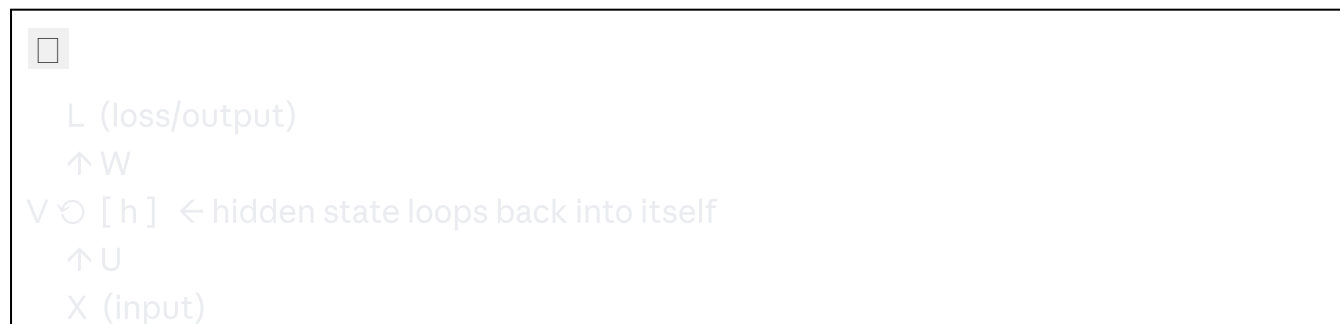
3. The Recurrent Neuron / Recurrent Unit

The **recurrent unit** is the core building block of an RNN.

It maintains a **hidden state** that stores information about everything the network has seen in the sequence so far.

At each time step, this hidden state is fed back into the unit, letting the model retain and reuse past information — this is what allows it to capture **temporal dependencies**.

Diagrammatically (a single recurrent neuron, "rolled up" view):



4. Unrolling the RNN (Unfolding Through Time)

Because the same recurrent unit is reused at every time step, we can draw it "unrolled" — one copy of the network per time step, connected in a chain:



Why unroll?

It shows explicitly how information flows from one time step to the next.

It makes it possible to apply **Backpropagation Through Time (BPTT)** — errors are propagated backward across *all* time steps, not just one layer.

BPTT updates weights using information from the *entire* sequence, which is how the RNN learns temporal/sequential patterns.

Critical fact: All the unrolled copies share the **same weights** (U , V , W) and the **same bias** — it is *not* a new set of parameters per time step. This weight sharing is what lets an RNN handle sequences of arbitrary length with a fixed number of parameters.

Likely exam question: "Why do all time steps in an unrolled RNN share the same weights?"

Answer: Because it's conceptually the *same* recurrent unit being applied repeatedly over time (like a loop), not a stack of different units. Weight sharing keeps the parameter count independent of sequence length and lets the model generalize the same transition rule to every position in the sequence.

5. Hidden State and Output Calculations (The Core Math)

At every time step, an RNN applies a **recurrence formula**:

Hidden state:

$$a_t^h = U \cdot X_t + V \cdot h_{t-1} + B$$

$$h_t = \sigma(a_t^h)$$

Where:

$X_t \in \mathbb{R}^{d_x}$ — input at time t

$h_t \in \mathbb{R}^{d_h}$ — hidden state at time t

$U \in \mathbb{R}^{d_h \times d_x}$ — input-to-hidden weight matrix

$V \in \mathbb{R}^{d_h \times d_h}$ — hidden-to-hidden (recurrent) weight matrix

$B \in \mathbb{R}^{d_h}$ — bias

σ — activation function (e.g., tanh/sigmoid)

Output:

$$a_t^y = W \cdot h_t + C$$

$$\hat{y}_t = O(a_t^y)$$

Where:

$W \in \mathbb{R}^{d_y \times d_h}$ — hidden-to-output weight matrix

$C \in \mathbb{R}^{d_y}$ — output bias

O — output activation function (e.g., softmax for classification)

Loss:

$$L_t = \text{Loss}(\hat{y}_t, y_t), \quad L = \sum_t L_t$$

Calculation Summary (plain-English version):

The **same** computational function and **same** parameters (U, V, W, B, C) are used at every time step.

At each time step: take the current input **and** the previous hidden state $\rightarrow$ compute the current output.

The loss is typically computed at every time step, and the total loss is the sum over all time steps.

Likely exam question: "Write the equations for computing the hidden state and output of a vanilla RNN at time t ." **Answer:** $a_t^h = UX_t + Vh_{t-1} + B$, then $h_t = \sigma(a_t^h)$ $a_t^y = Wh_t + C$

C , then $\hat{y}_t = O(a_t^y)$

Likely exam question: "What does U , V , and W represent respectively in an RNN?" **Answer:** U = weight matrix from input to hidden state; V = weight matrix from previous hidden state to current hidden state (the recurrent connection); W = weight matrix from hidden state to output.

6. Training an RNN: Backpropagation Through Time (BPTT)

Principle: Unfold the computational graph across all time steps, then apply standard backpropagation across that unfolded graph. This variant is called **Backpropagation Through Time (BPTT)**. Once unfolded, any general-purpose gradient-based optimization technique (SGD, Adam, etc.) can be applied.

Gradient with respect to W (output weights)

Chain: $L \leftarrow L_t \leftarrow \hat{y}_t \leftarrow a_t^y \leftarrow W, C$

Error at output pre-activation:

$$\delta_t^y = \frac{\partial L_t}{\partial a_t^y} = \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial a_t^y} = \frac{\partial L_t}{\partial \hat{y}_t} \cdot O'(a_t^y) \in \mathbb{R}^{d_y}$$

Since $\frac{\partial a_t^y}{\partial W} = h_t$:

$$\frac{\partial L_t}{\partial W} = \delta_t^y h_t^\top \in \mathbb{R}^{d_y \times d_h}$$

Total gradient (summed over all time steps):

$$\frac{\partial L}{\partial W} = \sum_t \delta_t^y h_t^\top$$

Gradient with respect to U (input-to-hidden weights)

Chain: $X_t \rightarrow a_t^h$ (w.r.t. U, V) $\rightarrow h_t \rightarrow a_t^y \rightarrow \hat{y}_t \rightarrow L_t \rightarrow L$

Define the hidden error signal:

$$\delta_t^h = \frac{\partial L_t}{\partial a_t^h} = \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial a_t^y} \cdot \frac{\partial a_t^y}{\partial h_t} \cdot \frac{\partial h_t}{\partial a_t^h}$$

where the first three factors combine to $W^\top \delta_t^y$ (i.e. the error is passed backward through the output weight matrix).

Since $\frac{\partial a_t^h}{\partial U} = X_t$:

$$\frac{\partial L_t}{\partial U} = \delta_t^h X_t^\top \in \mathbb{R}^{d_h \times d_x} \Rightarrow \frac{\partial L}{\partial U} = \sum_t \delta_t^h X_t^\top$$

Gradient with respect to V (hidden-to-hidden / recurrent weights)

Same δ_t^h as above. Since $\frac{\partial a_t^h}{\partial V} = h_{t-1}$:

$$\frac{\partial L_t}{\partial V} = \delta_t^h h_{t-1}^\top \Rightarrow \frac{\partial L}{\partial V} = \sum_t \delta_t^h h_{t-1}^\top$$

Big picture takeaway: Because h_t depends on h_{t-1} , which depends on h_{t-2} , etc., the gradient w.r.t. V and U in particular involves a *product of many Jacobian/derivative terms* stretching back through time — this long chain of multiplications is exactly what causes the vanishing/exploding gradient problem (next section).

Likely exam question: "Why is training an RNN more complex than training a feedforward network?" **Answer:** Because the RNN must be "unrolled" across all time steps first, and gradients must be backpropagated through that entire unrolled chain (BPTT) — every time step's hidden state depends on all previous hidden states, so the gradient computation involves accumulating contributions across the whole sequence, not just one layer.

Likely exam question: "What is δ_t^h and why is it important?" **Answer:** It's the error/gradient signal at the hidden pre-activation at time t ($\partial L_t / \partial a_t^h$). It's important because it's reused to compute the gradients for both U and V, and it's propagated backward recursively through time via $V^\top$, which is the mechanistic source of vanishing/exploding gradients.

7. Vanishing & Exploding Gradients

Root cause: During backpropagation, gradients are computed by **multiplying many derivative terms** across time steps/layers (because of the chain rule through the unrolled recurrence). Repeated multiplication of many terms can make the product either shrink or grow extremely fast.

If the derivative terms have magnitude $< 1 \rightarrow$ repeated multiplication $\rightarrow$ gradient **shrinks exponentially** (vanishes).

If the derivative terms have magnitude $> 1 \rightarrow$ repeated multiplication $\rightarrow$ gradient **grows exponentially** (explodes).

Deep networks / long sequences $\rightarrow$ longer chains $\rightarrow$ the effect is stronger (more terms multiplied together).

Vanishing Gradients

Gradients become **near zero** by the time they reach early time steps/layers.

Earlier layers/time steps learn very slowly or effectively stop learning at all.

Practical consequence: the RNN **can't learn long-range dependencies** — it "forgets" things from far back in the sequence.

Exploding Gradients

Gradients become **very large**.

Leads to **unstable training**, weight values blowing up, and the loss diverging (e.g., turning into NaN).

Impact

Difficult optimization.

Poor convergence.

Likely exam question: *"Explain why vanishing gradients occur in RNNs."* **Answer:** Because computing the gradient at an early time step requires multiplying together many derivative terms (Jacobians), one per time step, via the chain rule (BPTT). If each of these terms has magnitude less than 1, their product shrinks exponentially as the number of time steps increases, so the gradient reaching earlier time steps becomes vanishingly small — those steps then learn very slowly or not at all.

Likely exam question: *"What's the practical difference in symptoms between vanishing and exploding gradients?"* **Answer:** Vanishing gradients $\rightarrow$ training stalls, early time steps stop learning, slow/no convergence. Exploding gradients $\rightarrow$ training becomes unstable, weights blow up, loss diverges/becomes NaN.

8. Variants of RNN

(a) Bidirectional Recurrent Neural Networks (BRNNs)

A BRNN processes the sequence in **both directions** — one "forward layer" going left→right, and one "backward layer" going right→left.

At each time step, the final representation **combines both past (forward) and future (backward) context**.

Motivating example: "They went to the bank to fish." — the meaning of the word "**bank**" (river bank vs. financial bank) only becomes clear once you see what comes **after** it ("fish"). A standard (unidirectional) RNN, which only sees past context, couldn't resolve this ambiguity at the time it processes "bank"; a BRNN can, because it also incorporates future context via the backward pass.

Structure: separate forward hidden states $\overleftarrow{h}_t$ and backward hidden states $\overrightarrow{h}_t$ are combined to produce the output y_t at each step.

Likely exam question: "Why would you use a bidirectional RNN instead of a standard RNN?"

Answer: When the meaning/label at a given position depends on **both** past and future context (not just what came before), a standard RNN — which only has access to past information — will be insufficient. A BRNN processes the sequence in both directions and merges past and future context, e.g., disambiguating "bank" in "They went to the bank to fish."

(b) Long Short-Term Memory Networks (LSTMs)

LSTMs introduce a **dedicated memory cell** (the cell state) that helps mitigate the vanishing gradient problem.

Each LSTM cell has **three gates** controlling information flow:

Input Gate — how much new information gets written into the cell state.

Forget Gate — selectively removes information from the past that's no longer needed.

Output Gate — decides what part of the cell state is exposed as the current output/hidden state.

This controlled memory system lets LSTMs retain important information over **long sequences** — making them effective for tasks needing long-range context.

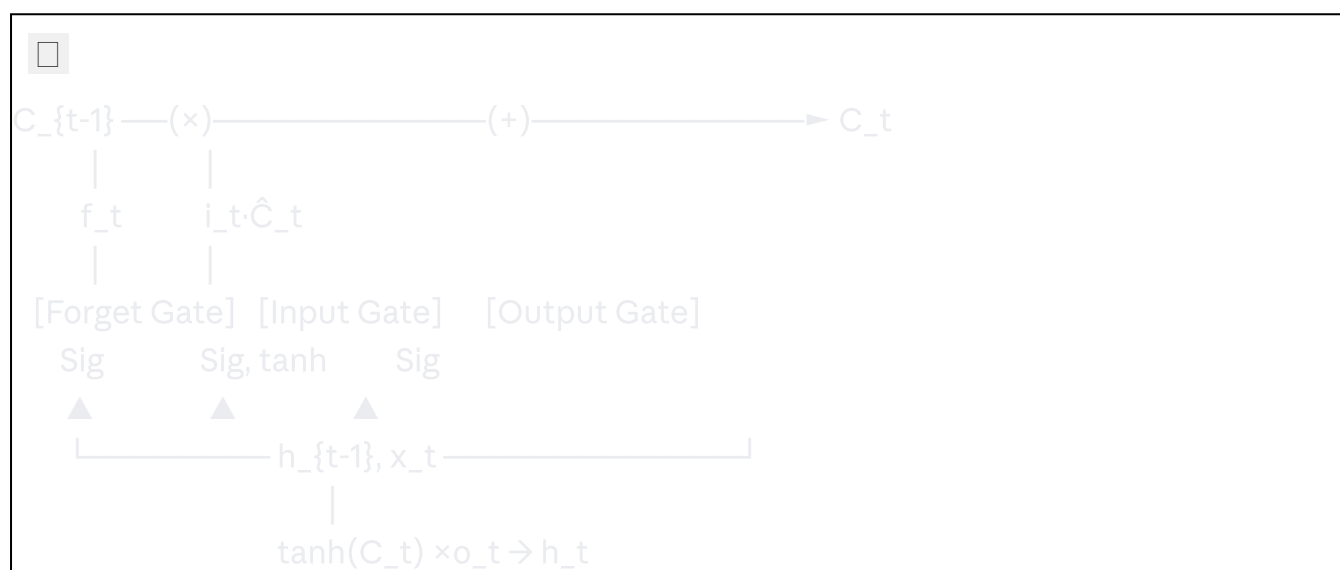
9. LSTM — Full Architecture

The LSTM cell has two "streams" running through it:

Cell state C_t — the long-term memory, running mostly untouched (only linear interactions: multiply and add) along the top of the diagram.

****Hidden state h_t **** — the short-term output/working memory, also passed to the next time step and often used as the output at time t .

Diagram (conceptually):



$\otimes$ = point-by-point (element-wise) multiplication

$\oplus$ = point-by-point (element-wise) addition

Sig = sigmoid function (squashes to $[0,1]$ — acts like a "gate" or filter)

$\tanh$ = tanh function (squashes to $[-1,1]$ — used to create/scale candidate content)

Cell State C — the LSTM's Memory

C_t is the LSTM's internal memory, designed to store **long-term dependencies**.

It carries information across many time steps with **minimal modification** (this is precisely why it avoids vanishing gradients — gradients can flow through mostly-linear paths).

The three gates **add, remove, or expose** parts of this memory:

Forget Gate decides what portion of C_{t-1} should be **kept**.

Input Gate decides how much **new** information should be added.

Output Gate uses the cell state to determine the **hidden state**.

Likely exam question: "Why does the LSTM cell state help with vanishing gradients while the plain RNN hidden state doesn't?" **Answer:** The cell state is updated mostly through element-wise (linear) operations — multiplying by the forget gate and adding the input

contribution — rather than being repeatedly passed through a weight matrix and nonlinearity at every step like the plain RNN's hidden state. This creates a more direct, largely uninterrupted path for gradients to flow backward through time, so gradients don't have to be multiplied by many small numbers repeatedly (which is what causes vanishing).

10. LSTM Gates in Detail

10.1 Forget Gate — *"What old info should be erased?"*

Works like a **cleanup tool** that routinely removes outdated or irrelevant memories. If past information is no longer useful, this gate reduces its influence on the cell state. Prevents the memory from being cluttered with unnecessary details.

Mechanism: Takes two inputs — x_t (current input) and h_{t-1} (previous hidden state/output) — multiplies each by its own weight matrix, adds a bias, then passes the sum through a **sigmoid** (output range $[0, 1]$):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

If the output for a given cell-state component is **0 (or near 0)** → that piece of information is **forgotten**.

If the output is **1 (or near 1)** → that information is **retained** for future use.

10.2 Input Gate — *"What new info should be stored?"*

Acts like a **filter** deciding whether new information is important enough to remember. If the current input is meaningful, the input gate "opens" to let it into memory; if not, it "closes" to keep noise out.

Mechanism: Two parts are computed from h_{t-1} and x_t :

The **gate** value (how much to let in), via sigmoid:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

The ****candidate**** new content, via tanh (scaled to $[-1, 1]$):

$$\hat{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

These combine (element-wise multiply $i_t \odot \hat{C}_t$) to determine how much of the new candidate information actually gets added to the cell state.

Cell state update (combining forget + input gates — shown in the architecture diagram as the $\otimes$ then $\oplus$ operations along the top):

$$C_t = f_t \odot C_{t-1} + i_t \odot \hat{C}_t$$

(i.e., keep the fraction f_t of the old memory, and add the fraction i_t of the new candidate memory.)

10.3 Output Gate — "*What should be revealed right now?*"

Functions like a **spotlight** deciding which part of the stored memory is relevant for the current step.

Selects the information that should influence the output at this moment.

Keeps the internal memory and the final output appropriately aligned.

Mechanism:

First, a sigmoid decides which parts of the cell state to expose, based on h_{t-1} and x_t :

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

Next, the (already-updated) cell state C_t is passed through **tanh** to rescale its values to $[-1, 1]$.

Finally, this is multiplied element-wise by o_t to produce the hidden state h_t — **which is also the output of this LSTM cell** at this time step:

$$h_t = o_t \odot \tanh(C_t)$$

Likely exam question: "Write the full set of LSTM equations for one time step." **Answer:** $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ (forget gate) $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$ (input gate) $\hat{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$ (candidate memory) $C_t = f_t \odot C_{t-1} + i_t \odot \hat{C}_t$ (cell state update) $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$ (output gate) $h_t = o_t \odot \tanh(C_t)$ (hidden state / output)

Likely exam question: "What is the role of the sigmoid function vs. the tanh function in an LSTM?" **Answer:** Sigmoid (range $[0, 1]$) is used in all three **gates** (forget, input, output) because it acts as a soft "how much to let through" switch — 0 means block completely, 1 means pass completely. Tanh (range $[-1, 1]$) is used to create/rescale the **content** itself

(the candidate cell values $\hat{C}_t$, and again when producing h_t from C_t) since it can represent both positive and negative contributions, centered at zero.

Likely exam question: "Both the forget gate and input gate use a sigmoid on $[h_{t-1}, x_t]$ — what's the actual difference between them?" **Answer:** They have different learned weight matrices (W_f vs. W_i) and serve opposite purposes: the forget gate's output (f_t) is multiplied against the **old** cell state C_{t-1} to decide what to keep/discard from the past, while the input gate's output (i_t) is multiplied against the **new candidate** $\hat{C}_t$ to decide how much new information to add.

11. How LSTM Gates Control Vanishing/Exploding Gradients

Forget Gate → prevents exploding gradients. It decides how much of the previous memory to keep. If old/unneeded information is present, the forget gate reduces its influence, preventing the memory from accumulating unbounded/useless information that could otherwise make gradients explode.

Input Gate → prevents vanishing gradients. It controls how much new information enters the memory — important information is let in strongly, unimportant information is weakened. This keeps meaningful signals strong enough that they don't fade away over time.

Controlled Gradient Flow: Gates use sigmoid and tanh functions, which **bound** their outputs to fixed ranges ($[0,1]$ and $[-1,1]$ respectively). This bounding keeps values — and therefore gradients flowing through them — within stable, controlled ranges, rather than letting them multiply unboundedly across time steps as in a vanilla RNN.

Likely exam question: "Which LSTM gate is primarily responsible for combating vanishing gradients, and which for exploding gradients?" **Answer:** Input gate → primarily combats **vanishing** gradients (keeps important new signals strong). Forget gate → primarily combats **exploding** gradients (prevents unnecessary accumulation of memory).

12. Vanilla RNN vs. LSTM — Summary Comparison

Aspect	Vanilla RNN	LSTM
Memory	Remembers only the recent	Remembers what truly matters — both

Aspect	Vanilla RNN	LSTM
	past (short-term)	short- and long-term
Forgetting behavior	Forgets quickly (no explicit control)	Knows explicitly what to keep and what to throw away (via gates)
Memory type	Short-term memory only	Short-term and long-term memory (hidden state + cell state)
Behavior	Reacts to the immediate moment	Understands the whole "story"/context
Time horizon	Recalls roughly "yesterday"	Recalls the whole timeline
Long sequences	Struggles (vanishing gradients)	Carries important information forward effectively

Likely exam question: *"In one or two sentences, why do LSTMs outperform vanilla RNNs on long sequences?"* **Answer:** LSTMs add a separate cell state with gated (forget/input/output) control over what information is kept, added, or exposed, allowing gradients and important information to flow across many time steps largely undistorted — whereas a vanilla RNN's single hidden state is repeatedly overwritten through a weight matrix and nonlinearity every step, causing it to lose long-range information and suffer from vanishing gradients.

13. Quick-Fire Concept Checklist (for last-minute revision)

- ☐ Can you list all 5 sequence-processing setups (one-to-one, one-to-many, many-to-one, many-to-many ×2) with an example each?
- ☐ Can you state, in words, what makes an RNN different from a feedforward net?
- ☐ Can you write the hidden-state and output equations of a vanilla RNN from memory?
- ☐ Do you know why the weights are shared across time steps in the unrolled view?
- ☐ Can you explain BPTT in 2–3 sentences?
- ☐ Can you derive/state $\partial L / \partial W$, $\partial L / \partial U$, $\partial L / \partial V$ conceptually (chain rule through time)?
- ☐ Can you explain *why* vanishing/exploding gradients happen (repeated multiplication of derivative terms)?

- ☐ Can you explain a Bidirectional RNN and give the "bank" example?
- ☐ Can you name and explain the purpose of all three LSTM gates?
- ☐ Can you write the six core LSTM equations ($f_t, i_t, \hat{C}_t, C_t, o_t, h_t$)?
- ☐ Can you explain which gate helps with vanishing vs. exploding gradients, and why?
- ☐ Can you give at least 3 differences between vanilla RNN and LSTM?

14. Extra Broad/Conceptual Questions Likely to Appear

Q: What problem in vanilla RNNs motivated the invention of LSTMs? A: Vanilla RNNs suffer from vanishing (and exploding) gradients during BPTT because gradients must be propagated through many repeated multiplications across time steps. This makes it very hard for vanilla RNNs to learn long-range/long-term dependencies. LSTMs were designed specifically to solve this by introducing a gated cell state that preserves information (and gradient flow) over long sequences.

Q: In the LSTM diagram, what do the $\otimes$ and $\oplus$ symbols represent, and where do they appear? A: $\otimes$ represents element-wise (point-by-point) multiplication — used, e.g., when the forget gate scales the old cell state, and when the input gate scales the candidate values, and when the output gate scales $\tanh(C_t)$. $\oplus$ represents element-wise addition — used to combine the "kept" old memory with the "new" candidate memory to form the updated cell state C_t .

Q: What are the dimensions of the key RNN parameters, given input dimension d_x , hidden dimension d_h , and output dimension d_y ? A: $U \in \mathbb{R}^{d_h \times d_x}$, $V \in \mathbb{R}^{d_h \times d_h}$, $B \in \mathbb{R}^{d_h}$, $W \in \mathbb{R}^{d_y \times d_h}$, $C \in \mathbb{R}^{d_y}$ (bias), $h_t \in \mathbb{R}^{d_h}$, $\hat{y}_t \in \mathbb{R}^{d_y}$.

Q: What does it mean that the RNN "shares parameters across time"? Why is that useful? A: It means the exact same U , V , W (and biases) are reused at every time step rather than learning a fresh set per position. This is useful because it (1) keeps the number of parameters fixed regardless of sequence length, (2) lets the model generalize a learned pattern to any position in the sequence, and (3) makes the model able to handle variable-length sequences.

Q: Where is the loss computed in an RNN, and how is total loss defined? A: A loss L_t is typically computed at every time step by comparing the predicted output $\hat{y}_t$ to the true target y_t . The total loss L for the whole sequence is the sum of the per-time-step losses: $L = \sum_t L_t$.

Q: What is the intuitive purpose of the output gate's use of $\tanh$ on the cell state? A: The cell state C_t can, in principle, take on large or unbounded magnitude values as it

accumulates information. Passing it through $\tanh$ rescales it to a bounded range of $[-1, 1]$ before it's exposed as part of the output, keeping the hidden state numerically well-behaved.

Q: True/False style — "In an LSTM, the hidden state and the cell state are the same thing." A: False. The cell state C_t is the long-term memory that flows through with minimal modification; the hidden state h_t is derived from the cell state via the output gate ($h_t = o_t \odot \tanh(C_t)$) and represents the "exposed"/working output at time t , also passed to the next step.

Good luck with your exam! This guide covers every concept, diagram, formula, and framing phrase found across all 25 slides of Lecture 15.